Zero to Hero Study Handbook: temporal-graph-aml-intelligence

This handbook is built from static analysis of the repository files only.

Module 1: Foundations & Architecture

1.1 What this project does

This project builds an anti-money-laundering (AML) risk-intelligence workflow on the Elliptic Bitcoin transaction graph. It compares graph and non-graph models under a strict temporal split (past for training, future for testing), then evaluates outputs in analyst-operations terms such as Recall@K and Precision@K.

Main use cases implemented in this repo:

- Temporal graph assembly from Elliptic CSVs into tensors for model training (`src/data_loader.py`).
- Supervised node-risk modeling with GraphSAGE and GAT (`src/models.py`, `src/train.py`).
- Tabular baselines and anomaly scoring in a notebook workflow (`generate_notebook.py` -> `temporal_graph_aml_intelligence.ipynb`).
- Hybrid risk scoring and analyst queue simulation (notebook cells generated by `generate_notebook.py`).

1.2 Core paradigms and patterns used in this repository

- Temporal ML split pattern: The split is by `time_step`, not random. `build_graph(train_max_ts=30, val_max_ts=34)` creates masks where test is strictly future (`time_step > val_max_ts`). This enforces no future leakage.
- Reusable module + notebook orchestration: Core logic is in `src/` modules, while `generate_notebook.py` composes a narrative notebook that imports these modules.
- Transductive full-batch GNN training: `train_gnn()` uses full graph tensors and mask-based supervised loss (`train_mask`) in one forward pass per epoch.
- Hybrid modeling pattern: The notebook combines supervised scores (GNN + XGBoost) with unsupervised anomaly scores (Isolation Forest), then builds both weighted and learned blends.
- Functional utility style around ML primitives: `src/train.py` exposes pure-style metric helpers (`evaluate_scores`, `metrics_by_time`, `recall_at_k`, `precision_at_k`) that consume arrays and return dictionaries/lists.
- Light OOP where state is natural: `EllipticData` dataclass encapsulates graph tensors and split masks; GNNs are `nn.Module` classes (`GraphSAGE`,

GAT).

1.3 Architecture and component interaction

High-level components:

- Data ingest and graph assembly: `src/data_loader.py`
- GNN model definitions: `src/models.py`
- Training and metrics: `src/train.py`
- Notebook generation and orchestration: `generate_notebook.py`
- Executed study artifact: `temporal_graph_aml_intelligence.ipynb`
- Output artifacts: `figures/*.png`, `figures/summary_metrics.csv`, `checkpoints/*.pt`

Main flow (ASCII diagram):

```
Raw Elliptic CSVs (data/raw/elliptic/elliptic_bitcoin_dataset)
        |
        v
load_elliptic_frames() -> build_graph() -> EllipticData
        |                                |
        |                                +-> tabular_split() for sklearn/xgboost baselines
        |
        +-> to_undirected() for GNN message passing
                        |
                        v
                  build_model('sage'|'gat')
                        |
                        v
                    train_gnn()
                        |
                        +-> gnn_predict() -> probabilities + embeddings
                                             |
                       +---------------------+------------------+
                       |                                        |
                       v                                        v
              evaluate_scores()/metrics_by_time()      IsolationForest on features/embeddi
                       |                                        |
                       +---------------------+------------------+
                                             v
                                Hybrid scores (weighted + meta)
                                             |
                                             v
                                  Analyst queue simulation + plots/csv
```

Module 2: Repository Map

The table below focuses on files a new contributor should understand first.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`README.md`	Project overview, objective, dataset instructions, run commands, and expected results framing.	N/A	Data download commands; temporal split narrative (`train <=30`, `val 31..34`, `test >=35`).
`pyproject.toml`	Python project metadata and dependencies.	N/A	`requires-python = ">=3.12.10"`; dependencies (`torch`, `torch-geometric`, `xgboost`, `scikit-learn`, etc.); `tool.uv.sources.torch` index override to `pytorch-cu128`.
`.python-version`	Local Python version pin for tooling.	N/A	`3.12.10`.
`generate_notebook.py`	Programmatically builds `temporal_graph_aml_intelligence.ipynb` from markdown/code cells. Defines full tutorial workflow steps 0-11.	`md()`, `code()`	`SEED = 42`; notebook-level paths `FIG = Path("figures")`, `CKPT = Path("checkpoints")`; saved outputs (`figures/*.png`, `figures/summary_metrics.csv`, `checkpoints/{arch}_best.pt`).
`temporal_graph_aml_intelligence.ipynb`	Executed learner deliverable containing end-to-end experiment narrative, code, and outputs.	Imports and uses `build_graph`, `build_model`, `train_gnn`, `gnn_predict`, `evaluate_scores`, etc.	Runtime constants in notebook cells: `SEED`, `POS_WEIGHT`, `GNN='sage'`; device selection via `pick_device(min_free_gb=2.5)`.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
src/data_loader.py	Elliptic CSV loading, label mapping, graph tensor construction, temporal masks, tabular split utilities.	EllipticData, load_elliptic_frames(), build_graph(), to_undirected(), tabular_split()	DEFAULT_ROOT = Path("data/raw/elliptic/elliptic_bitcoin_ label constants ILLICIT=1, LICIT=0, UNKNOWN=-1; defaults train_max_ts=30, val_max_ts=34.
src/models.py	GNN architectures and model factory.	GraphSAGE, GAT, build_model()	Model hyperparameters: hidden, num_layers, dropout, heads (GAT); final head outputs single logit per node.
src/train.py	Seed control, ranking metrics, temporal metrics, GNN training loop, inference helper.	set_seed(), recall_at_k(), precision_at_k(), evaluate_scores() metrics_by_time() TrainConfig, train_gnn(), gnn_predict()	TrainConfig keys: epochs, lr, weight_decay, pos_weight, patience, log_every; early stopping tracked by best validation PR-AUC.
figures/summary_metrics.csv	Persisted consolidated benchmark metrics from notebook execution.	N/A	Columns: model, PR-AUC, ROC-AUC, R@100, P@100, R@500, P@500.
.gitignore	Prevents committing generated or local artifacts.	N/A	Ignores .venv/, data/, checkpoints/*.pt, figures/*.png, etc.

Module 3: Core Execution Flows

Flow A: Notebook generation pipeline

Entrypoint: generate_notebook.py

Step-by-step:

1. Helper constructors create notebook cells:

```
def md(src: str) -> nbf.NotebookNode: ...
def code(src: str) -> nbf.NotebookNode: ...
```

2. A `cells` list is assembled with ordered sections (`## 1` to `## 11`).
3. Notebook metadata is set (`kernelspec`, `language_info`).
4. The file is written to:

```
out = Path(__file__).parent / "temporal_graph_aml_intelligence.ipynb"
```

Input/Output shape:

- Input: none besides source code in `generate_notebook.py`.
- Output: one `.ipynb` JSON file with `cells` and notebook metadata.

Flow B: Data loading and graph construction

Primary functions: `load_elliptic_frames()` -> `build_graph()` -> optional `to_undirected()` and `tabular_split()`.

1. `load_elliptic_frames(root)` reads:

- `elliptic_txs_features.csv`
- `elliptic_txs_classes.csv`
- `elliptic_txs_edgelist.csv`

2. It renames feature columns:

- `txId`, `time_step`, then `feat_0 ... feat_164`.

3. `build_graph()` maps labels:

```
label_map = {"1": 1, "2": 0, "unknown": -1}
```

4. It remaps `txId` to contiguous node indices `0..N-1`.
5. It creates tensors:

- `x: torch.Tensor` shape `[N, F]` (`float32`)
- `y: torch.Tensor` shape `[N]` (`int64`, values in `{1,0,-1}`)
- `time_step: torch.Tensor` shape `[N]` (`int64`)
- `edge_index: torch.Tensor` shape `[2, E]` (`long`)

6. It computes masks on labelled nodes only (`y != -1`):

- `train_mask: ts <= train_max_ts`
- `val_mask: train_max_ts < ts <= val_max_ts`
- `test_mask: ts > val_max_ts`

7. `to_undirected(edge_index)` concatenates reversed edges, returning shape `[2, 2E]`.

8. `tabular_split(data)` returns a dict:

```
{
  "train": (X_train, y_train, time_train),
  "val":   (X_val, y_val, time_val),
  "test":  (X_test, y_test, time_test)
}
```

with `X_*` as NumPy arrays from `data.x` and label/time arrays aligned to mask positions.

Flow C: Model build, train, and infer

Primary modules: `src/models.py`, `src/train.py`

1. Build model via factory:

```
model = build_model("sage", in_dim=data.num_features, hidden=128, num_layers=2, dropout=0.3)
# or "gat"
```

2. Configure training with `TrainConfig`.

TrainConfig fields and meaning:

- `epochs`: max training epochs.
- `lr`: Adam learning rate.
- `weight_decay`: L2 regularization.
- `pos_weight`: class-imbalance weighting for positive class in BCE-with-logits.
- `patience`: early-stopping window on validation PR-AUC.
- `log_every`: logging frequency.

3. `train_gnn(model, data, device, cfg)`:

- Uses full-batch forward pass over all nodes.
- Computes loss on `train_mask` only.
- Evaluates PR-AUC on `val_mask` each epoch.
- Tracks and returns best validation checkpoint.

Return values:

- `best_state`: `dict[str, torch.Tensor]` (state dict snapshot).
- `history`: `list[dict]` with keys `epoch`, `loss`, `val_pr_auc`.

4. `gnn_predict(model, data, device, mask)` returns:

- `p`: `np.ndarray` probabilities for nodes in `mask`.
- `e`: `np.ndarray` embeddings from `model.embed(...)` for same nodes.

Flow D: Evaluation, anomaly branch, hybrid scoring, and analyst queue

Implemented in notebook code cells generated by `generate_notebook.py`.

1. Baseline models are fit on tabular split:

- `LogisticRegression`
- `RandomForestClassifier`
- `xgb.XGBClassifier`

2. Anomaly branch:

- `IsolationForest` on raw features and on GNN embeddings.
- Risk score uses negated `score_samples`.

3. Hybrid scoring:

- Weighted blend of rank-normalized signals (`gnn`, `xgb`, `anom`).
- Meta-model (`LogisticRegression`) trained on validation signal matrix.

4. Metric evaluation with `evaluate_scores(y_true, scores)`.

`evaluate_scores` output dictionary shape:

```
{
  "pr_auc": float,
  "roc_auc": float,
  "n": int,
  "n_pos": int,
  "recall@50": float,
  "precision@50": float,
  "recall@100": float,
  "precision@100": float,
  "recall@200": float,
  "precision@200": float,
  "recall@500": float,
  "precision@500": float
}
```

(Exact `@K` keys depend on the `ks` argument, default `(50, 100, 200, 500)`).

5. Temporal diagnostics:

- `metrics_by_time(y_true, scores, time_steps)` returns `list[dict]` rows with keys:
  - `time_step`, `n`, `n_illicit`, `pr_auc`, `roc_auc`

6. Persisted outputs:

- `figures/summary_metrics.csv`
- PNG plots in `figures/`
- Checkpoints in `checkpoints/` (`sage_best.pt`, `gat_best.pt`)

Module 4: Setup & Run Guide

4.1 Prerequisites

- OS/tooling: Linux-compatible workflow, `uv` package manager.
- Python: `3.12.10` (from `.python-version` and `pyproject.toml`).
- Notebook tooling: `jupyter`, `ipykernel` (declared dependencies).
- Optional GPU path: Torch wheels pulled from `pytorch-cu128` index (`pyproject.toml`).

4.2 Dependency install (clean machine)

```
git clone https://github.com/pypi-ahmad/temporal-graph-aml-intelligence.git
cd temporal-graph-aml-intelligence
uv sync
```

4.3 Data prerequisites

Expected raw dataset location by default:

`data/raw/elliptic/elliptic_bitcoin_dataset`

Download command from `README.md`:

```
uv run kaggle datasets download -d ellipticco/elliptic-data-set \
    -p data/raw/elliptic --unzip
```

Optional benchmark dataset command in `README.md`:

```
uv run kaggle competitions download -c ieee-fraud-detection -p data/raw/ieee
unzip data/raw/ieee/ieee-fraud-detection.zip -d data/raw/ieee
```

4.4 Environment variables and credentials

This repo does not include a `.env` template file. Required external credential setup is for Kaggle access:

- Either Kaggle token file at `~/.kaggle/kaggle.json`
- Or environment token (as documented in `README.md`): `KAGGLE_API_TOKEN`

No other environment-variable keys are defined in source files.

4.5 Typical command sequences

Generate notebook from script:

`uv run python generate_notebook.py`

Open notebook interactively:

`uv run jupyter lab temporal_graph_aml_intelligence.ipynb`

Headless notebook execution:

```
uv run jupyter nbconvert --to notebook --execute --inplace \
    temporal_graph_aml_intelligence.ipynb
```

4.6 Data migrations or seeding

- No database layer exists in this repository.
- No migrations, schema migration tools, or seed scripts are present.
- Data prep is file-based CSV loading only.

Module 5: Study Plan & Practice Exercises

5.1 Recommended study order

1. `README.md` Learn project objective, dataset assumptions, and run modes.
2. `pyproject.toml` + `.python-version` Understand runtime constraints and dependency stack.
3. `src/data_loader.py` Master the data model first (label mapping, masks, edge construction).
4. `src/models.py` Understand the GNN architectures and what `embed()` exposes.
5. `src/train.py` Understand training loop, early stopping, and metric utilities.
6. `generate_notebook.py` See how all components are orchestrated into a complete analytical workflow.
7. `temporal_graph_aml_intelligence.ipynb` Read the executed narrative and compare with generator script.
8. `figures/summary_metrics.csv` Inspect concrete benchmark output and reconcile with pipeline logic.

5.2 Practice exercises (with solution outlines)

1. Exercise: Trace how class labels are transformed from raw CSV values to training labels.

- Target files: `src/data_loader.py`
- What to find: mapping of `'1'`, `'2'`, `'unknown'`; how missing labels are handled.
- Solution outline: `label_map = {"1": 1, "2": 0, "unknown": -1}`; merge on `txId`; missing values filled with `UNKNOWN=-1`.

2. Exercise: Explain why unknown nodes remain in the graph but are excluded from loss/metrics.

- Target files: `src/data_loader.py`, `src/train.py`

- Solution outline: Unknown nodes can contribute graph structure and message passing signal. Masks (`train_mask`, `val_mask`, `test_mask`) are built on `labelled = y != UNKNOWN`, and training loss uses only `logits[train_mask]`.

3. Exercise: Describe the exact tensor shapes produced by `build_graph()`.

- Target files: `src/data_loader.py`
- Solution outline: `x [N,F]`, `edge_index [2,E]`, `y [N]`, `time_step [N]`, plus boolean masks `[N]`.

4. Exercise: Compare GraphSAGE and GAT implementation choices in this codebase.

- Target files: `src/models.py`
- Solution outline: GraphSAGE uses `SAGEConv` + ReLU + LayerNorm; GAT uses `GATConv` with multi-head attention and ELU. Both end with linear head returning one logit per node and expose `embed()`.

5. Exercise: Explain the early-stopping condition in `train_gnn()`.

- Target files: `src/train.py`
- Solution outline: After each epoch, compute validation PR-AUC. If improved, snapshot state dict and reset counter. Otherwise increment `since`. Stop when `since >= cfg.patience`.

6. Exercise: Reconstruct the dictionary keys emitted by `evaluate_scores()` and why they suit AML operations.

- Target files: `src/train.py`
- Solution outline: Keys include global rank metrics (`pr_auc`, `roc_auc`) and queue metrics (`recall@K`, `precision@K`) for operational review-budget framing.

7. Exercise: Identify where hybrid scoring is built and list the input signals.

- Target files: `generate_notebook.py`
- Solution outline: In section `## 7`. Signals are GNN probability (`gnn`), XGBoost probability (`xgb`), and anomaly score (`anom`). Two hybrids: weighted rank blend and logistic meta-model.

8. Exercise: Show where persisted outputs are produced and what each artifact means.

- Target files: `generate_notebook.py`, `figures/summary_metrics.csv`
- Solution outline: `torch.save(...)` writes GNN checkpoints; `summary.to_csv('figures/summary_metrics.csv')` writes consolidated scores; multiple `plt.savefig(...)` write diagnostic figures.

9. Exercise: Explain how device selection avoids accidental GPU contention.

- Target files: `generate_notebook.py` / notebook imports cell.

- Solution outline: `pick_device(min_free_gb=2.5)` checks CUDA availability and free memory (`torch.cuda.mem_get_info()`); falls back to CPU if insufficient free GPU memory.

10. Exercise: Explain what has to change if dataset files are stored in a non-default location.

- Target files: `src/data_loader.py`
- Solution outline: Pass `root` explicitly into `load_elliptic_frames(root=...)` or `build_graph(root=...)`; otherwise default is `DEFAULT_ROOT`.

Understanding Checklist

Use this checklist before saying you understand the repository end-to-end:

- Can you explain why the temporal split in `build_graph()` prevents future leakage?
- Can you describe every field in `EllipticData` and where it is used later?
- Can you trace model construction from `build_model()` to `train_gnn()` to `gnn_predict()`?
- Can you explain why the project keeps both tabular and graph baselines in the same workflow?
- Can you list the output keys from `evaluate_scores()` without looking them up?
- Can you explain how the hybrid score is formed (weighted and meta-model variants)?
- Can you locate where summary metrics and plots are persisted?
- Can you describe the full command path from clone to notebook execution on a clean machine?
- Can you identify which external credential is required and where it is used?
- Can you justify the main design tradeoff in this codebase: modular logic in `src/` with notebook-driven orchestration?